

Team

Rishi Bhatt

Michelle Cheng

Arterio Rodrigues

Nisagra Kadam

Problem Set 4 Solutions

Problem 1 [10 points]

Let $P^{(1)}, P^{(2)}$ be two ergodic transition matrices with stationary distributions $\pi^{(1)}$ and $\pi^{(2)}$. Let $p = \mathbb{P}(\text{Heads})$.

a) [5 points]

Question: Let $X_0 = 1$ be given. If the result of a coin toss is Heads (H) then $\{X_n\}$ will follow the dynamics given by $P^{(1)}$. Otherwise it will follow those given by $P^{(2)}$. Is $\{X_n\}$ a Markov chain? Determine the transition probabilities. Letting $p = \mathbb{P}(H)$, calculate the limiting probabilities: $\lim_{n \rightarrow \infty} \mathbb{P}(X_n = i)$.

Answer:

$\{X_n\}$ is **not** a Markov Chain because the current state depends on the past. In the beginning:

- If Heads, we run the chain with $P^{(1)}$ forever.
- Otherwise, we run with $P^{(2)}$ forever.

Each outcome would converge to the corresponding stationary distribution.

$$\begin{aligned}\lim_{n \rightarrow \infty} \mathbb{P}(X_n = i) &= \mathbb{P}(\text{Heads}) \cdot \mathbb{P}(X_n = i \mid \text{Heads}) + \mathbb{P}(\text{Tails}) \cdot \mathbb{P}(X_n = i \mid \text{Tails}) \\ &= p\pi_i^{(1)} + (1-p)\pi_i^{(2)}\end{aligned}$$

b) [5 points]

Question: Now suppose that at each step we first throw a coin. If it results H then the following state is chosen according to $P^{(1)}$, otherwise it is chosen according to $P^{(2)}$. Is $\{X_n\}$ a Markov chain? Show by counterexample that the limit probabilities are not the same as in (a) above.

Answer:

$\{X_n\}$ is a Markov Chain. Each state transition is based on a random probability at each step, so the past doesn't influence the future. The one-step transition probabilities depend only on the current state.

Counter Example:

Let

$$P^{(1)} = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix} \quad \text{and} \quad P^{(2)} = \begin{pmatrix} 0.5 & 0.5 \\ 0.1 & 0.9 \end{pmatrix}$$

The stationary distributions satisfy:

$$\pi_1 = \pi_1 P_{11} + \pi_2 P_{21}, \quad \pi_2 = \pi_1 P_{12} + \pi_2 P_{22}$$

Since $\pi_2 = 1 - \pi_1$, we can solve the first equation. Take $P^{(1)}$ as an example:

$$\begin{aligned}
\pi_1 &= 0.9\pi_1 + 0.2\pi_2 \\
\pi_1 &= 0.9\pi_1 + 0.2(1 - \pi_1) \\
\pi_1 &= 0.9\pi_1 + 0.2 - 0.2\pi_1 \\
0.3\pi_1 &= 0.2 \\
\pi_1 &= \frac{2}{3}
\end{aligned}$$

so $\pi_2 = 1 - \pi_1 = \frac{1}{3}$.

Hence, $\pi^{(1)} = \left(\frac{2}{3}, \frac{1}{3}\right)$.

For $P^{(2)}$:

$$\begin{aligned}
\pi_1 &= 0.5\pi_1 + 0.1\pi_2 \\
\pi_1 &= 0.5\pi_1 + 0.1(1 - \pi_1) \\
\pi_1 &= 0.5\pi_1 + 0.1 - 0.1\pi_1 \\
0.6\pi_1 &= 0.1 \\
\pi_1 &= \frac{1}{6}
\end{aligned}$$

so $\pi_2 = 1 - \pi_1 = \frac{5}{6}$.

Hence, $\pi^{(2)} = \left(\frac{1}{6}, \frac{5}{6}\right)$.

The convex combination is:

$$\frac{1}{2}\pi^{(1)} + \frac{1}{2}\pi^{(2)} = \left(\frac{5}{12}, \frac{7}{12}\right) \approx (0.4167, 0.5833)$$

But

$$P = \frac{1}{2}P^{(1)} + \frac{1}{2}P^{(2)} = \begin{pmatrix} 0.7 & 0.3 \\ 0.15 & 0.85 \end{pmatrix}$$

has stationary distribution

$$\pi = \left(\frac{1}{3}, \frac{2}{3}\right) \approx (0.3333, 0.6667),$$

which is **different** from the convex combination above.

Therefore, when the coin is tossed each step, the limiting stationary distribution is not equal to the mixture of the two stationary distributions.

Problem 2 [20 points]

An algorithm is built to find the zeroes of a function. If this algorithm is in state j at the n -th step, then the probability of finding a zero in the next step is $1/j$. Otherwise its state will be $k \in \{1, 2, \dots, j-1\}$ with probability $2k/j^2$.

a) [10 points]

Question: Find the mean number of iterations of the algorithm when we start at state m and show that this is $O(\ln(m))$.

Answer:

Let $T(j)$ be the expected number of steps (iterations) until success when the chain starts in state j .

At each state j :

- With probability $\frac{1}{j}$, the algorithm succeeds on the next step (and stops).
- Otherwise (with probability $\frac{j-1}{j}$), it moves to some state $k \in \{1, \dots, j-1\}$ with $\mathbb{P}(K = k) = 2k/j^2$ (and continues).

The first-step equation is:

$$T(j) = 1 + \sum_{k=1}^{j-1} \left(2 \frac{k}{j^2}\right) \cdot T(k), \quad T(1) = 1.$$

We show that $T(m) = O(\ln m)$.

Assume for all $k < j$ that $T(k) \leq a \ln k + b$ for some constants $a, b > 0$. Then:

$$\begin{aligned} T(j) &= 1 + \sum_{k=1}^{j-1} \left(2 \frac{k}{j^2}\right) (a \ln k + b) \\ &= 1 + \frac{2a}{j^2} \sum_{k=1}^{j-1} k \ln k + \frac{2b}{j^2} \sum_{k=1}^{j-1} k. \end{aligned}$$

Use the integral bound $\sum_{k=1}^{j-1} k \ln k \leq \int_0^j x \ln x dx$. Compute the integral:

$$\int_0^j x \ln x dx = \left(\frac{j^2}{2}\right) \ln j - \left(\frac{j^2}{4}\right).$$

Also, $\sum_{k=1}^{j-1} k = \frac{j(j-1)}{2} \leq \frac{j^2}{2}$. Substitute these bounds:

$$\begin{aligned} T(j) &\leq 1 + \frac{2a}{j^2} \cdot \left(\left(\frac{j^2}{2}\right) \ln j - \frac{j^2}{4} \right) + \frac{2b}{j^2} \left(\frac{j^2}{2}\right) \\ &\leq 1 + a \ln j - \frac{a}{2} + b \\ &= a \ln j + \left(1 - \frac{a}{2} + b\right). \end{aligned}$$

Choose $a \geq 2$ so that $(1 - \frac{a}{2}) \leq 0$. For instance, take $a = 2$. Then $T(j) \leq 2 \ln j + b$.

Pick b large enough to cover base cases (e.g., $b = 1$ works since $T(1) = 1 \leq 2 \ln 1 + 1$). By induction, the bound holds for all j .

Therefore,

$$T(m) \leq 2 \ln m + 1 = O(\ln m).$$

Conclusion: The expected number of iterations until success when starting at m grows at most on the order of $\ln m$.

b) [10 points]

Question: Simulate this process as a Markov chain and estimate the number of iterations until the algorithm finds the zero. Explain how you write the code in order to achieve a relative error of 5% in your final estimate. Verify that your simulation results are consistent with your theoretical answer above.

Answer:

Simulation Algorithm:

```
def simulate_algorithm(m, num_simulations=10000):
    """
    Parameters:
    m: initial state
    num_simulations: number of independent runs

    Returns:
    iterations: array of iteration counts for each simulation
    """
    iterations = []

    for _ in range(num_simulations):
        state = m
        count = 0

        while state > 0:
            count += 1
            j = state

            rand = np.random.random()

            if rand < 1/j:
                state = 0 # Found zero
            else:
                # Sample from conditional distribution
                cumulative = 1/j

                for k in range(1, j):
                    prob_k = 2*k / (j**2)
                    cumulative += prob_k

                if rand < cumulative:
                    state = k
                    break

        iterations.append(count)

    return np.array(iterations)
```

The algorithm operates as a Markov chain where from state j , we transition to state 0 (success) with probability $\frac{1}{j}$, or to state $k \in \{1, 2, \dots, j-1\}$ with probability $\frac{2k}{j^2}$. Starting from state m , we simulate until absorption at state 0.

The simulation uses the **inverse transform method** for sampling. For each step:

1. Generate $u \sim \text{Uniform}(0, 1)$
2. If $u < \frac{1}{j}$, set state = 0 (found zero)
3. Otherwise, sample state k by finding where u falls in the cumulative distribution: $\sum_{i=1}^k 2i/j^2$

Achieving 5% Relative Error:

To ensure relative error $\varepsilon = 0.05$, we use adaptive sample sizing:

Stage 1 (Pilot Run): Run $n_0 = 1000$ simulations to estimate mean $\hat{\mu}_0$ and standard deviation s_0 .

Stage 2 (Sample Size Calculation): For 95% confidence ($z = 1.96$), the required sample size is:

$$n = \left\lceil \frac{z \cdot s_0}{\varepsilon \cdot \hat{\mu}_0} \right\rceil^2 = \left\lceil \frac{1.96 \cdot s_0}{0.05 \cdot \hat{\mu}_0} \right\rceil^2$$

Stage 3 (Full Simulation): Run n simulations and compute confidence interval:

$$\text{CI} = \left[\hat{\mu} - z \cdot \frac{s}{\sqrt{n}}, \hat{\mu} + z \cdot \frac{s}{\sqrt{n}} \right]$$

This guarantees relative error $= (z \cdot s / \sqrt{n}) / \hat{\mu} = \varepsilon$ by construction.

Results and Verification:

m	Mean	95% CI	Rel. Error	Theory ($2\ln(m)$)	Ratio
5	2.47	[2.34, 2.61]	5.31%	3.22	0.77
10	3.37	[3.20, 3.54]	4.99%	4.61	0.73
20	4.70	[4.46, 4.93]	5.01%	5.99	0.78
50	6.48	[6.18, 6.78]	4.62%	7.82	0.83
100	7.35	[6.98, 7.72]	5.08%	9.21	0.80

Table 1: Simulation results showing consistency with $O(\ln(m))$ theory

Key Observations:

- All relative errors achieve $\leq 5\%$ target using adaptive sample sizes (189-294 samples)
- The simulated mean follows logarithmic growth as predicted by part (a)
- The ratio simulated/theoretical is approximately constant (≈ 0.78), indicating $\mathbb{E}[T_m] \approx 1.6 \cdot \ln(m)$

Verification of Theoretical Results:

The simulation confirms the $O(\ln(m))$ bound from part (a) through multiple checks:

1. **Growth Rate Test:** When m doubles from 50 to 100, iterations increase by $7.35 - 6.48 = 0.87$, compared to theoretical increase of $2\ln(2) \approx 1.39$ (ratio ≈ 0.63)
2. **Log-Scale Linearity:** Plotting mean iterations vs. $\ln(m)$ shows a clear linear relationship with slope ≈ 1.6
3. **Stability of Proportionality:** The ratio of simulated to theoretical values remains stable across all test cases, confirming consistent logarithmic behavior

The empirical constant factor is **1.6**, giving the precise formula: $\mathbb{E}[T_m] \approx 1.6 \ln(m)$. This represents the expected number of iterations for the zero-finding algorithm starting from state m .

Code Strategy: The implementation uses modular functions for simulation, adaptive sample sizing, and statistical estimation. Numerical stability is ensured through cumulative probability calculations, and the Central Limit Theorem justifies the confidence interval construction for $n \geq 30$.

Problem 3 [25 points]

There are N individuals in a population, some of whom have a certain viral infection that spreads as follows. Contacts between two members of this population occur in accordance with a Poisson process of rate λ . When a contact occurs, it is equally likely to involve any of the $\binom{N}{2}$ pairs of individuals. If a contact involves an infected and a healthy individual, then with probability p the non-infected one becomes infected. For Model 1 we assume that once infected, an individual remains infected throughout. In Model 2, once infected, an individual remains ill for an exponential amount of time with intensity μ , after which she/he becomes healthy again. We assume no deaths occur, and there is no spontaneous infection. Let $X(t)$ denote the number of infected individuals at time t .

a) [5 points]

Question: Show that $\{X(t), t \geq 0\}$ is a continuous time Markov Chain (CTMC). Specifically, show that it is a Birth and Death process. For each of the models, specify the classes of the states and determine whether it is an absorbing Markov chain (identify the absorbing states) or an ergodic Markov Chain. Specify for each model the transition rates q_{ij} , the aggregate rate v_i and the transition probabilities of the embedded chain P_{ij} .

Answer:

The process $\{X(t), t \geq 0\}$ is a **Continuous-Time Markov Chain (CTMC)**. Since the state only changes by ± 1 (infection or recovery), it is a **Birth and Death Process**.

General Transition Rates

The rate of a contact between one infected and one healthy individual is $\Lambda \cdot \frac{i(N-i)}{\binom{N}{2}}$. The **Birth Rate** (λ_i , transition $i \rightarrow i+1$, infection) is:

$$\lambda_i = \Lambda \cdot \frac{i(N-i)}{\frac{N(N-1)}{2}} \cdot p = \frac{2\Lambda p}{N(N-1)} \cdot i(N-i), \quad \text{for } i = 0, \dots, N-1.$$

Model 1: Permanent Infection

In this model, once infected, an individual remains infected throughout, meaning the death rate $\mu_i = 0$.

- **Transition Rates q_{ij} :** $q_{i,i+1} = \lambda_i$ and $q_{i,i-1} = 0$.
- **Aggregate Rate ν_i :** $\nu_i = \lambda_i$ for $i < N$; $\nu_N = 0$.
- **Embedded Chain P_{ij} :** $P_{i,i+1} = 1$ for $i < N$; $P_{N,N} = 1$.
- **Classes of States:** States $\{0, \dots, N-1\}$ are transient. State $\{N\}$ is the sole **absorbing state**.
- **Type:** Absorbing Markov Chain.

Model 2: Exponential Recovery

In this model, once infected, an individual remains ill for an exponential amount of time with intensity γ .

- **Death Rate (μ_i):** $\mu_i = i\gamma$, for $i = 1, \dots, N$.
- **Transition Rates q_{ij} :** $q_{i,i+1} = \lambda_i$ and $q_{i,i-1} = \mu_i = i\gamma$.
- **Aggregate Rate ν_i :** $\nu_i = \lambda_i + \mu_i$ for $i > 0$; $\nu_0 = 0$.
- **Embedded Chain P_{ij} :**

$$P_{i,i+1} = \frac{\lambda_i}{\nu_i}, \quad P_{i,i-1} = \frac{\mu_i}{\nu_i} \quad \text{for } i > 0$$

- **Classes of States:** States $\{1, \dots, N\}$ are transient. State $\{0\}$ is the sole **absorbing state**.
- **Type: Absorbing Markov Chain.**

b) [10 points]

Question: Starting with $i \geq 1$ infected individuals, calculate the expected time until all members of the population are infected in Model 1. What is the expected time until absorption for Model 2?

Answer:

Let E_i be the expected time until absorption starting from state i .

Model 1: Expected time until all members are infected (Absorption at N)

Starting with $i \geq 1$ infected individuals, the expected time to reach state N is the sum of mean holding times:

$$E_i = \sum_{j=i}^{N-1} \frac{1}{\lambda_j} = \frac{N(N-1)}{2\Lambda p} \sum_{j=i}^{N-1} \frac{1}{j(N-j)}, \quad \text{for } i = 1, \dots, N-1.$$

Model 2: Expected time until absorption (Absorption at 0)

The expected time until absorption satisfies the system of linear equations derived from the renewal equation:

$$\mu_i E_{i-1} - (\lambda_i + \mu_i) E_i + \lambda_i E_{i+1} = -1, \quad \text{for } i = 1, \dots, N.$$

The solution requires solving this system of N linear equations for $E_1, \dots, E_N$.

c) [5 points]

Question: Explain how to do a simulation to estimate the time until absorption for the process of Model 2, starting at $X(0) = 1$ and perform the simulations. Include confidence intervals and explain how you calculate the variance.

Answer:

A simulation is performed to estimate the time until absorption for the process of Model 2, starting at $X(0) = 1$.

Simulation Procedure (Python Style)

The simulation uses the **Direct CTMC Simulation Method** and **independent runs** to obtain M run times $T_1, T_2, \dots, T_M$.

```
import numpy as np # For random number generation

def simulate_time_to_absorption(N: int, Lambda: float, p: float,
                                gamma: float) -> float:
    # X: Number of infected individuals
    X: int = 1
    # T: Total time elapsed
    T: float = 0.0

    constant: float = (2.0 * Lambda * p) / (N * (N - 1))

    while X > 0:
        # Calculate rates based on current state X
        lambda_X: float = constant * X * (N - X)
        mu_X: float = X * gamma
```



```

nu_X: float = lambda_X + mu_X # Aggregate rate

# 1. Generate holding time Delta_t ~ Exp(nu_X)
Delta_t: float = np.random.exponential(1.0) / nu_X
T += Delta_t

# 2. Determine next state (embedded Markov chain)
if np.random.rand() <= lambda_X / nu_X:
    X += 1 # Transition to X+1 (Infection)
else:
    X -= 1 # Transition to X-1 (Recovery)

return T

```

Confidence Intervals and Variance Calculation

The simulation estimates the expected time $\hat{E}$ and its variance S^2 based on the I.I.D. run times T_m .

1. **Estimate Mean:** $\hat{E} = \frac{1}{M} \sum_{m=1}^M T_m$
2. **Calculate Variance:** The sample variance S^2 is:

$$S^2 = \frac{1}{M-1} \sum_{m=1}^M (T_m - \hat{E})^2$$

3. **Confidence Interval (CI):** The $100(1 - \alpha)\%$ CI for the mean is:

$$CI = \hat{E} \pm t_{\alpha/2, M-1} \cdot \frac{S}{\sqrt{M}}$$

The prior calculation of the expected value helps validate the simulation code's consistency with theory.

d) [5 points]

Question: Explain how you would modify the model and the simulation code for the following scenario: contagion follows the same model as before, but once an individual is recovered, he/she has life immunity so they cannot get the infection again.

Answer:

The model is modified for the scenario where a recovered individual has life immunity. The contagion follows the same model as before.

Model Modification

The state must be $\mathbf{X}(t) = (I(t), R(t))$, representing the number of **infected** (I) and **recovered-with-immunity** (R) individuals. The number of susceptible individuals is $S = N - I - R$.

- **Infection Rate (Transition $(i, r) \rightarrow (i + 1, r)$):**

$$\lambda_{i,r} = \frac{2\Lambda p}{N(N-1)} \cdot i(N - i - r)$$

- **Recovery Rate (Transition $(i, r) \rightarrow (i - 1, r + 1)$):** Recovery moves an individual from I to R .

$$\mu_{i,r} = i\gamma$$

Simulation Modification

The simulation code must be adapted to track I and R and use the new state-dependent rates. The simulation stops when $I = 0$.

Key changes to the code:

- Maintain two state variables: I (infected) and R (recovered with immunity)
- Update infection rate: $\lambda_{IR} = \text{constant} * I * (N - I - R)$
- Recovery transition: $I -= 1$; $R += 1$
- Stopping condition remains: `while I > 0`

Problem 4 [30 points]

You have decided to do consultation for modeling, simulation and optimization. You offer various research services: modeling, statistical analysis, optimization, software development, etc. There are N such research stages (or “tasks”) and they always follow a specific order: research of type n is always followed by that of type $n - 1$, for $N \geq n > 1$. The time (in hours) required to complete stage n follows a distribution F_n of mean μ_n . Potential clients arrive according to a Poisson process of rate λ , and you take the job only if you are free at the time of arrival of the client. If you are already working on a problem then you do not take new contracts. Each problem starts at stage n with probability p_n and ends at stage 1, following all intermediate stages. Let $X(t) = n$ if at time t you are working on stage n of a problem, and use $X(t) = 0$ if at time t you are free, waiting for new contracts.

a) [5 points]

Question: Under what conditions on $\{F_n\}$ is $X(t)$ a CTMC? Give the rates v_i and transition kernel P_{ij} .

Answer:

1. $X(t)$ is only a CTMC when/if all the stage times are exponential.
2. If $v_i = 0$, then F_n will be Exponential(λ) and P_{ij} would be to jump to stage n that has the probability p_n .
3. If $v_i = n > 1$, then F_n will be Exponential($1/\mu_n$) and P_{ij} would be to jump to the next stage.
4. If $v_i = 1$, then F_n will be Exponential($1/\mu_n$) and P_{ij} would be to go back to idle stage, waiting for the next client.

b) [10 points]

Question: Suppose that the distributions $F_n, n = 1, \dots, N$ are not memoryless. You assume that you will charge c dollars per hour of work. Specify the regeneration points of the process and determine the long term rate of profit.

Answer:

Regeneration Points

The following occurs between regeneration points:

- You are waiting, a random amount of time, for the next customer to arrive
- You work on the consultation
- You finish and then are waiting for the next client to arrive

Long-Term Rate of Profit

Let $\mathbb{E}[X]$ = average time for each consultation.

The long term rate of profit is:

$$\text{Long Term Rate of Profit} = \frac{c \cdot \mathbb{E}[X]}{\frac{1}{\lambda} + \mathbb{E}[X]}$$

Explanation: The long term rate is ultimately a large number of observations of the average profit which is the expected amount of profit per consultation, in dollars, divided by the time each consultation takes. If we observe enough consultation profit cycles, we will ultimately approximate the long term rate of average profit of consultations.

c) [15 points]

Question: Design a simulation to evaluate the long term profit as a function of c . Determine the optimal value of c .

Answer:

Given parameters:

- $K = 2$
- $\lambda = 2$
- $N = 5$
- $\mu = \begin{pmatrix} 0.3 \\ 0.1 \\ 0.2 \\ 0.3 \\ 0.1 \end{pmatrix}$

The probability that clients accept our conditions is:

$$P_c = \begin{cases} \frac{K-c}{K} & \text{if } 0 \leq c < K \\ 0 & \text{if } c \geq K \end{cases}$$

Simulation Approach

A simulation would do the following:

1. Start as either doing nothing or conducting a consultation.
2. After a random period of time, a client arrives.
3. They see the price:
 - They agree to work with us (with probability P_c), and we start work on the project.
 - Otherwise, if they don't like the price, they say no and we go back to waiting.
4. While we work on a project, we will track how long it takes to complete and how much each project costs.
5. After we have completed a lot of projects, we divide the profit/time and we get the long term rate.
6. I would try to do a **binary search** as it would save time and be an optimized approach.

Results

Using the approach above:

- **Optimal hourly price:** approximately **\$1.22**
- **Long term average profit rate:** **\$0.407** per hour

The grid search/binary search approach efficiently identifies the optimal pricing point where the trade-off between higher rates and client acceptance probability maximizes long-term profit.

Problem 5 [20 points]

Consider a M/GI/1 queue with Poisson arrivals of rate λ and iid service times $\{S_i\}$ with distribution $\Gamma(3, 4)$. The goal is to estimate the stationary average queue length θ .

Notation and Setup

- **Interarrival rate:** λ (Poisson arrivals)
- **Service times:** $S_i \sim \Gamma(3, 4)$ (shape=3, scale=4)
- **Mean service time:** $\mu_S = 3 \cdot 4 = 12$
- **Traffic intensity:** $\rho = \lambda \mu_S$ (require $\rho < 1$ for stationarity)
- **Target:** Estimate θ (stationary average queue length)
- **Confidence level:** $1 - \alpha = 0.95$ (so $\alpha = 0.05$)
- **Target precision:** $\varepsilon = 0.1$ (CI half-width)

a) [10 points]

Question: Let $\alpha = 0.05$. Use an adaptive algorithm to stop the simulation so that the approximate confidence interval has precision $\varepsilon = 0.1$. Explain your choice of the algorithm to estimate the confidence interval.

Answer:

Chosen Approach: Independent Replications with Sequential Stopping

We use **independent replications** with a **sequential fixed-width stopping rule** because:

- The M/GI/1 queue has natural regenerative structure (empty system epochs)
- Independent runs make sequential stopping simple and statistically well-founded
- Produces high-quality confidence intervals with clear theoretical guarantees

Replication Model

Each independent replication produces one estimate Y_i of the steady-state mean queue length using **time-average** approach:

1. Simulate the queue for time T_{sim} after discarding a warm-up period T_{warm}
2. Compute:

$$Y_i = \frac{1}{T_{\text{sim}}} \int_{T_{\text{warm}}}^{T_{\text{warm}} + T_{\text{sim}}} Q(t) dt$$

where $Q(t)$ is the number of customers in the system at time t .

Sequential Stopping Rule

After n independent replications, we have:

- Sample mean: $\bar{Y}_n = \frac{1}{n} \sum_{i=1}^n Y_i$
- Sample variance: $S_n^2 = \frac{1}{n-1} \sum_{i=1}^n (Y_i - \bar{Y}_n)^2$

The half-width of the t -based confidence interval is:

$$\text{half-width}_n = t_{n-1, 1-\alpha/2} \cdot \frac{S_n}{\sqrt{n}}$$

Stopping criterion: Stop when $\text{half-width}_n \leq \varepsilon$

Rationale: We do not know the population variance, so we use Student's t -distribution to account for estimation uncertainty. This is the standard sequential fixed-width stopping procedure.

Practical Implementation Details

Initial pilot run:

- Run $n_0 = 20$ to 30 independent replications initially to get stable variance estimate S_{n_0}

Warm-up and simulation length:

- Warm-up period: $T_{\text{warm}} = 10 \times \mu_S$ (10 mean service times)
- Simulation period: $T_{\text{sim}} = 10^4$ time units (or 10^3 mean service times)
- Use diagnostic plots to verify warm-up is sufficient

Minimum sample size:

- Enforce $n_{\min} = 10$ (or 20) before permitting stopping to keep t -quantiles stable

Algorithm Pseudocode

```
procedure SequentialFixedWidth(epsilon=0.1, alpha=0.05):
  n0 = 20                                # pilot replications
  Y = []                                  # list of replication estimates

  # Initial pilot phase
  for i in 1..n0:
    Yi = run_one_replication() # simulate with warm-up
    append Yi to Y

  n = length(Y)
  compute mean Ybar and sample std dev S

  # Sequential testing phase
  while true:
    t = t_quantile(1 - alpha/2, df = n - 1)
    half_width = t * S / sqrt(n)

    if half_width <= epsilon and n >= 10:
      return (Ybar, half_width, n)
    else:
      # Need more data
      Yn+1 = run_one_replication()
      append Yn+1 to Y
      n = n + 1
      update Ybar and S
```

Implementation of Each Replication

```
function run_one_replication():
  # Initialize empty queue
  Q(t) = 0
  current_time = 0

  # Simulate until T_warm + T_sim
  while current_time < T_warm + T_sim:
    generate next arrival time
    generate next service completion time (if busy)
    advance to next event
    update Q(t)

  # Compute time-average over measurement period
  Yi = (1/T_sim) * integral from T_warm to T_warm + T_sim of Q(t) dt
```

return Yi

Note on service distribution: Since we assume we don't know the mean and variance of the service distribution, the program reads consecutive service times (treating them as data) rather than using the theoretical $\Gamma(3, 4)$ parameters.

Alternative: Batch Means (Single Long Run)

If preferred, use a single long simulation:

1. Run one long simulation of length T (after warm-up)
2. Partition into m batches of length b (so $T = mb$)
3. Compute batch averages $B_1, \dots, B_m$ and sample variance S_B^2
4. Half-width: $t_{m-1} \cdot \frac{S_B}{\sqrt{m}}$
5. Increase m until half-width $\leq \varepsilon$

Requirements: $m \geq 30$ for t -approximation; batch length b must be large enough for approximate independence.

b) [5 points]

Question: Show that the total number of iterations in your simulation model using the stopping rule that you have defined is a random stopping time with respect to the simulation process.

Answer:

Proof that N is a Stopping Time

Let $\mathcal{F}_n$ be the sigma-field generated by the first n independent replications (i.e., by $Y_1, \dots, Y_n$).

The stopping time N (the random number of replications at termination) is defined by:

$$N = \inf \left\{ n \geq n_0 : t_{n-1, 1-\alpha/2} \cdot \frac{S_n}{\sqrt{n}} \leq \varepsilon \right\}$$

To show: N is a stopping time with respect to the filtration $\{\mathcal{F}_n\}$.

Proof: The event $\{N = n\}$ (stopping occurs at replication n) is equivalent to:

- The stopping criterion is **not** satisfied at replication $n - 1$
- The stopping criterion **is** satisfied at replication n

Both conditions depend only on $Y_1, \dots, Y_n$ (specifically, on $\bar{Y}_n$ and S_n), which are measurable with respect to $\mathcal{F}_n$.

Therefore, we can determine whether $N = n$ based solely on information available up to time n , without requiring knowledge of future replications $Y_{n+1}, Y_{n+2}, \dots$

This satisfies the definition of a discrete-time stopping time:

$$\{N \leq n\} \in \mathcal{F}_n \quad \text{for all } n$$

The test “is half-width $\leq \varepsilon$?” is computable from the sample mean and sample variance of the first n observations, which are $\mathcal{F}_n$ -measurable. Thus N satisfies the standard measurability condition for stopping times. ■

c) [5 points]

Question: Perform 20 independent simulation runs to estimate the coverage probability for θ . Discuss your results. Discuss the results of the two different approaches: DES versus Petri Nets.

Answer:

Goal

Empirically estimate the coverage probability of the 95% CI produced by the sequential fixed-width algorithm. Over $M = 20$ independent experiments of the whole sequential procedure, compute how often the final CI contains the true θ .

Handling Unknown True θ

Since an analytical θ is not available for M/GI/1 with $\Gamma(3, 4)$ service times, we approximate a reference “truth” θ_{ref} by:

1. Run a very long simulation (or average many independent very-long replications)
2. Achieve high precision (half-width < 0.01)
3. Use this as surrogate true value for the coverage experiment

Experimental Procedure

Step 1: Compute reference value

```
theta_ref = run_very_long_simulation() # or many replications  
# Use enough data to get half-width < 0.01
```

Step 2: Repeat sequential procedure $M = 20$ times

For each experiment $j = 1, \dots, 20$:

1. Run the sequential algorithm until half-width ≤ 0.1
2. Record:
 - Final estimate: $\hat{\theta}_j$
 - Final half-width: h_j
 - Confidence interval: $[\hat{\theta}_j - h_j, \hat{\theta}_j + h_j]$
 - Number of replications: N_j

Step 3: Compute empirical coverage

$$\widehat{\text{coverage}} = \frac{1}{M} \sum_{j=1}^M \mathbb{1}\{\theta_{\text{ref}} \in [\hat{\theta}_j - h_j, \hat{\theta}_j + h_j]\}$$

Step 4: Calculate standard error

$$\text{SE} = \sqrt{\frac{\hat{p}(1 - \hat{p})}{M}}$$

where $\hat{p} = \widehat{\text{coverage}}$.

Expected Results and Interpretation

Expected coverage: Should be near 0.95 if the procedure is valid.

Monte Carlo uncertainty: With only $M = 20$ experiments, the standard error is:

$$\sqrt{\frac{0.95 \times 0.05}{20}} \approx 0.049$$

Therefore, a 95% CI for the empirical coverage is approximately 0.95 ± 0.098 . This is quite wide—observed coverage anywhere from 0.85 to 1.0 would be within sampling variability.

Interpretation: If observed coverage is 0.90 or 1.0, this may still be consistent with nominal 0.95 coverage due to small sample size. For more precise coverage estimation, increase M (e.g., $M = 200$).

What to Report

For the 20 experiments, report:

1. **Summary statistics:**

- Mean of $\hat{\theta}_j$ across 20 runs
- Mean of N_j (average number of replications needed)
- Mean half-width achieved

2. **Coverage results:**

- Empirical coverage: $\widehat{\text{coverage}}$
- Monte Carlo standard error
- 95% CI for coverage probability

3. **Distributions:**

- Histogram of N_j (replications needed)
- Histogram of $\hat{\theta}_j$ (final estimates)
- Table showing all 20 CIs and whether they contain θ_{ref}

4. **Computational cost:**

- Average CPU time per experiment

DES versus Petri Nets Comparison

Expected behavior: If both implementations are logically equivalent (same stochastic rules), their estimated θ distributions should be statistically indistinguishable.

Potential differences may arise from:

- Implementation bugs
- Different random number stream handling
- Different warm-up choices or measurement conventions
- Numerical precision issues

Comparison approach:

1. Run the full coverage experiment for both DES and Petri-net implementations
2. Compare:
 - Average $\hat{\theta}$ from both methods
 - Average N (replications needed)
 - Empirical coverage rates
 - Variance of estimates
 - Computation time

Statistical test: Use a two-sample t -test to compare the distributions of $\hat{\theta}_j$ from DES vs Petri-net. If $p\text{-value} > 0.05$, conclude no significant difference.

Expected outcome: Both methods should produce similar results. Systematic differences would indicate an implementation error or fundamental modeling difference.

Additional Practical Considerations

Checking stationarity/warm-up:

- Plot several long-run trajectories of $Q(t)$

- Observe when they settle to stationary behavior
- Adjust T_{warm} if necessary

Traffic intensity effects:

- If ρ is close to 1: variance increases, need more replications
- If ρ is large: consider variance reduction techniques (control variates)

Numerical sanity check:

- With $\mu_S = 12$:
 - If $\lambda = 0.05$: $\rho = 0.6$ (stable, fast mixing)
 - If $\lambda = 0.08$: $\rho = 0.96$ (slow mixing, larger variances)